

TRABAJO PRÁCTICO 5: RELACIONES UML 1 A 1



Resolución de los ejercicios:

Ejercicio 1: Pasaporte – Foto – Titular

Relaciones:

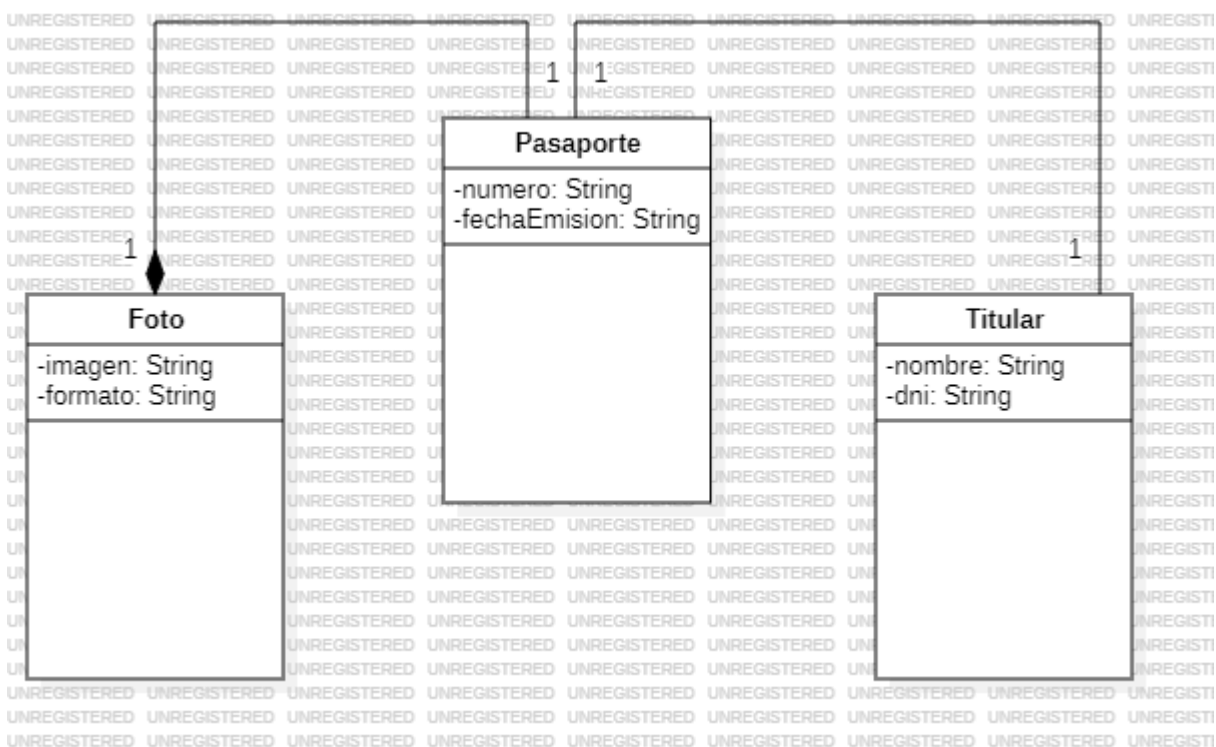
- **Composición:** Pasaporte → Foto
- **Asociación bidireccional:** Pasaporte ↔ Titular

Explicación breve:

La clase Pasaporte contiene una Foto en relación de composición, ya que la Foto depende del ciclo de vida del Pasaporte. Además, Pasaporte y Titular están relacionados mediante una asociación bidireccional porque ambos objetos se conocen mutuamente.

Diagrama y código:

Diagrama UML en StarUML:



Código en java:

```
// Clase Foto
public class Foto {
    private String imagen;
    private String formato;

    public Foto(String imagen, String formato) {
```

```

        this.imagen = imagen;
        this.formato = formato;
    }

    @Override
    public String toString() {
        return "Foto [imagen=" + imagen + ", formato=" + formato + "]";
    }
}

// Clase Titular
public class Titular {
    private String nombre;
    private String dni;
    private Pasaporte pasaporte; // Asociación bidireccional

    public Titular(String nombre, String dni) {
        this.nombre = nombre;
        this.dni = dni;
    }

    public void setPasaporte(Pasaporte pasaporte) {
        this.pasaporte = pasaporte;
    }

    @Override
    public String toString() {
        return "Titular [nombre=" + nombre + ", dni=" + dni + "]";
    }
}

// Clase Pasaporte
public class Pasaporte {
    private String numero;
    private String fechaEmision;
    private Foto foto; // Composición
    private Titular titular; // Asociación bidireccional

    public Pasaporte(String numero, String fechaEmision, Foto foto, Titular titular) {
        this.numero = numero;
    }
}

```

```

        this.fechaEmision = fechaEmision;

        this.foto = foto;

        this.titular = titular;

        titular.setPasaporte(this); // vínculo bidireccional
    }

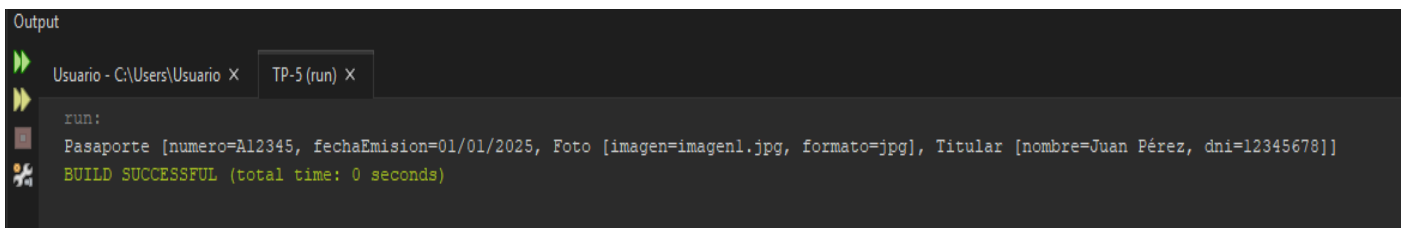
    @Override
    public String toString() {
        return "Pasaporte [numero=" + numero + ", fechaEmision=" + fechaEmision +
            ", " + foto + ", " + titular + "]";
    }
}

// main
public class Main {
    public static void main(String[] args) {
        Foto foto = new Foto("imagen1.jpg", "jpg");
        Titular titular = new Titular("Juan Pérez", "12345678");
        Pasaporte pasaporte = new Pasaporte("A12345", "01/01/2025", foto, titular);

        System.out.println(pasaporte);
    }
}

```

Ejecución en consola:



```

Output
Usuario - C:\Users\Usuario X  TP-5 (run) X
run:
Pasaporte [numero=A12345, fechaEmision=01/01/2025, Foto [imagen=imagen1.jpg, formato=jpg], Titular [nombre=Juan Pérez, dni=12345678]]
BUILD SUCCESSFUL (total time: 0 seconds)

```

Ejercicio 2: Celular – Batería – Usuario

Relaciones:

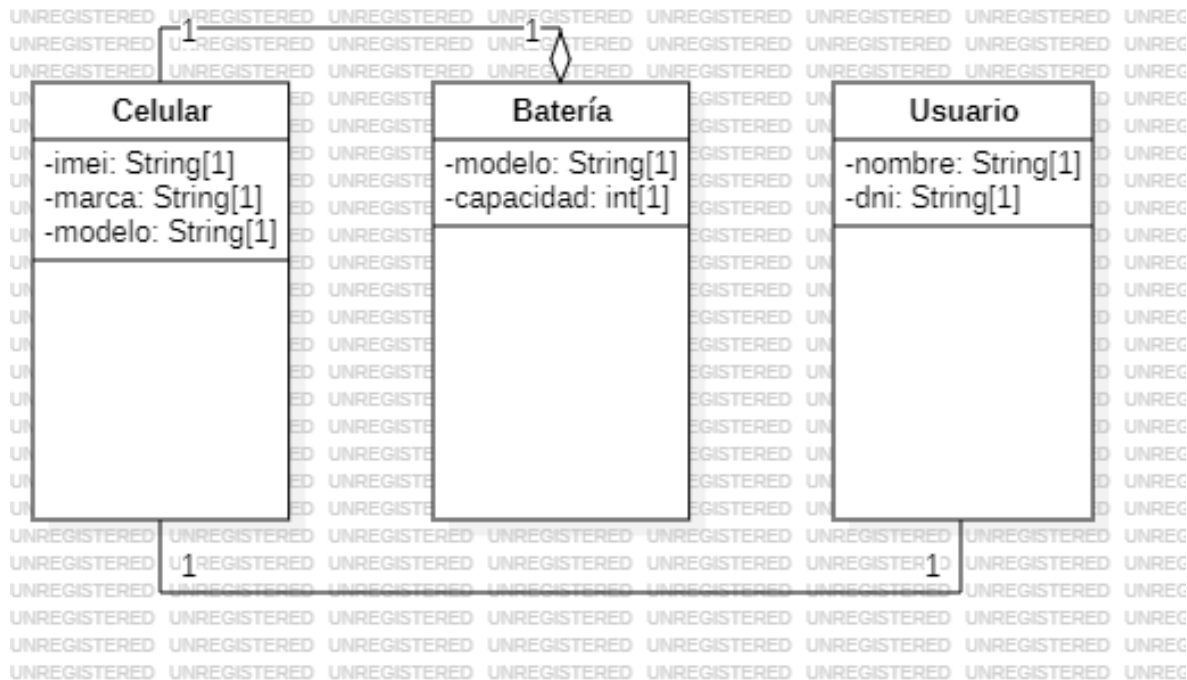
- Agregación: Celular → Batería
- Asociación bidireccional: Celular ↔ Usuario

Explicación breve:

El Celular se relaciona con la Batería mediante agregación, ya que la batería puede existir independientemente y ser utilizada en otro celular. Además, el Celular y el Usuario mantienen una asociación bidireccional, ya que ambos se conocen mutuamente.

Diagrama y código:

Diagrama UML en StarUML:



Código en Java:

```
// Clase Bateria
public class Bateria {
    private String modelo;
    private int capacidad; // en mAh

    public Bateria(String modelo, int capacidad) {
        this.modelo = modelo;
        this.capacidad = capacidad;
    }

    @Override
    public String toString() {
        return "Bateria [modelo=" + modelo + ", capacidad=" + capacidad + "mAh]";
    }
}
```

```
}
```

```
// Clase Usuario
```

```
public class Usuario {  
    private String nombre;  
    private String dni;  
    private Celular celular; // Asociación bidireccional  
  
    public Usuario(String nombre, String dni) {  
        this.nombre = nombre;  
        this.dni = dni;  
    }  
  
    public void setCelular(Celular celular) {  
        this.celular = celular;  
    }  
  
    @Override  
    public String toString() {  
        return "Usuario [nombre=" + nombre + ", dni=" + dni + "];"  
    }  
}
```

```
// Clase Celular
```

```
public class Celular {  
    private String imei;  
    private String marca;  
    private String modelo;  
    private Bateria bateria; // Agregación  
    private Usuario usuario; // Asociación bidireccional  
  
    public Celular(String imei, String marca, String modelo, Bateria bateria, Usuario usuario) {  
        this.imei = imei;  
        this.marca = marca;  
        this.modelo = modelo;  
        this.bateria = bateria;  
        this.usuario = usuario;  
        usuario.setCelular(this); // vínculo bidireccional  
    }  
}
```

```

@Override

public String toString() {
    return "Celular [imei=" + imei + ", marca=" + marca + ", modelo=" + modelo +
        ", " + bateria + ", " + usuario + "]\n";
}

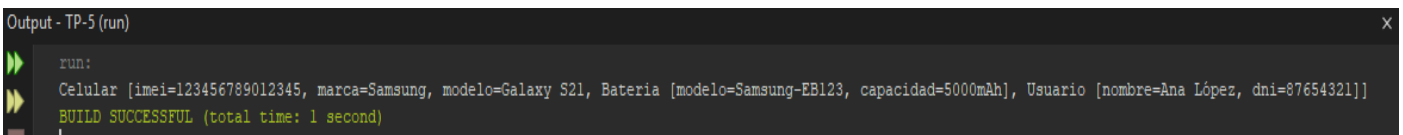
}

// main
public class Main {
    public static void main(String[] args) {
        Bateria bateria = new Bateria("Samsung-EB123", 5000);
        Usuario usuario = new Usuario("Ana López", "87654321");
        Celular celular = new Celular("123456789012345", "Samsung", "Galaxy S21", bateria, usuario);

        System.out.println(celular);
    }
}

```

Consola con la ejecución:



```

Output - TP-5 (run)
run:
Celular [imei=123456789012345, marca=Samsung, modelo=Galaxy S21, Bateria [modelo=Samsung-EB123, capacidad=5000mAh], Usuario [nombre=Ana López, dni=87654321]]
BUILD SUCCESSFUL (total time: 1 second)

```

Ejercicio 3: Libro – Autor – Editorial

Relaciones:

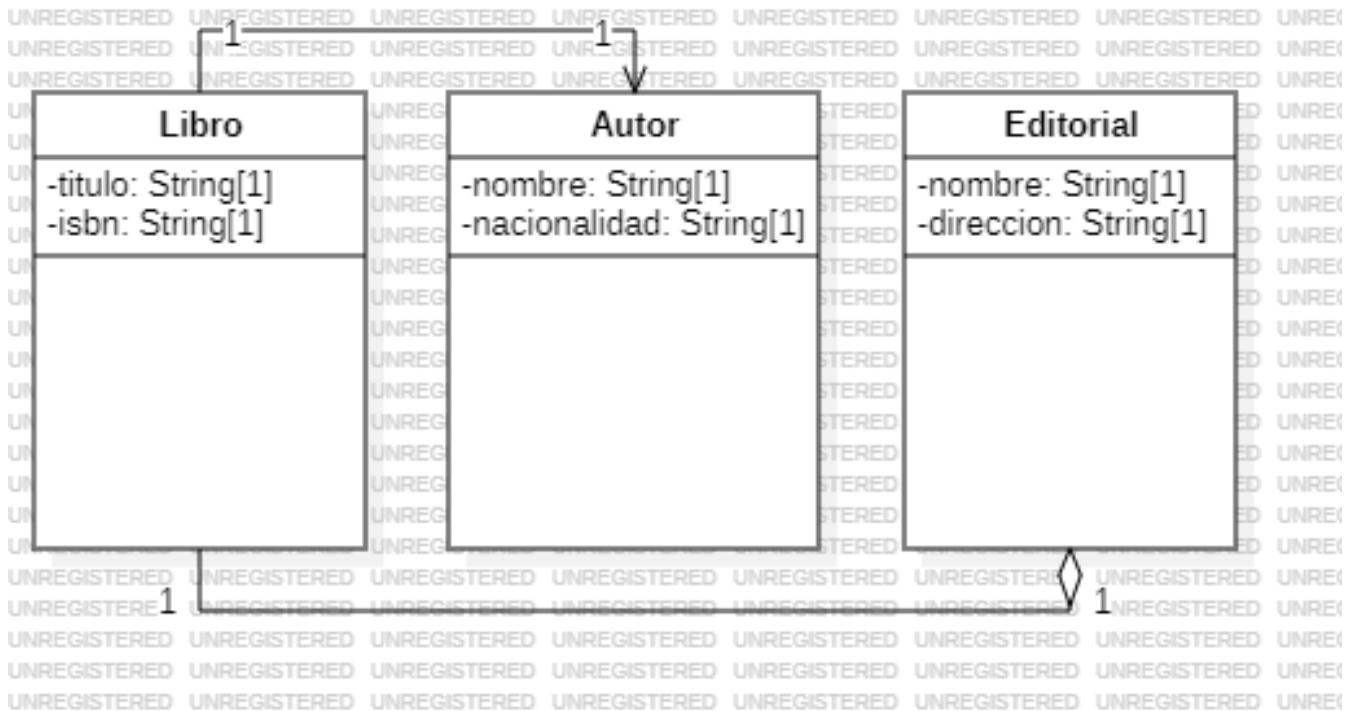
- Asociación unidireccional: Libro → Autor
- Agregación: Libro → Editorial

Explicación breve:

El Libro conoce a su Autor mediante una asociación unidireccional, ya que el Autor no necesita conocer a sus Libros. Además, el Libro tiene una relación de agregación con la Editorial, dado que la Editorial existe de manera independiente y puede estar asociada a múltiples libros.

Diagrama y código:

Diagrama UML en StarUML:



Código en Java:

```
// Clase Autor
public class Autor {
    private String nombre;
    private String nacionalidad;

    public Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }

    @Override
    public String toString() {
        return "Autor [nombre=" + nombre + ", nacionalidad=" + nacionalidad + "];"
    }
}

// Clase Editorial
public class Editorial {
    private String nombre;
    private String direccion;
```



```

public Editorial(String nombre, String direccion) {
    this.nombre = nombre;
    this.direccion = direccion;
}

@Override
public String toString() {
    return "Editorial [nombre=" + nombre + ", direccion=" + direccion + "]";
}
}

// Clase Libro
public class Libro {
    private String titulo;
    private String isbn;
    private Autor autor;          // Asociación unidireccional
    private Editorial editorial; // Agregación

    public Libro(String titulo, String isbn, Autor autor, Editorial editorial) {
        this.titulo = titulo;
        this.isbn = isbn;
        this.autor = autor;
        this.editorial = editorial;
    }

    @Override
    public String toString() {
        return "Libro [titulo=" + titulo + ", isbn=" + isbn +
            ", " + autor + ", " + editorial + "]";
    }
}

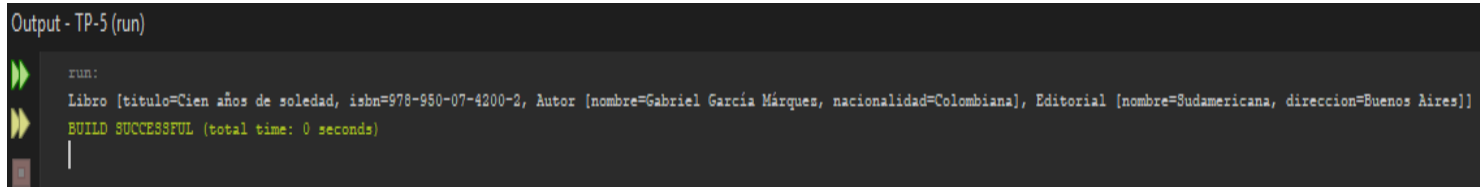
// main
public class Main {
    public static void main(String[] args) {
        Autor autor = new Autor("Gabriel García Márquez", "Colombiana");
        Editorial editorial = new Editorial("Sudamericana", "Buenos Aires");
        Libro libro = new Libro("Cien años de soledad", "978-950-07-4200-2", autor, editorial);

        System.out.println(libro);
    }
}

```

```
}  
}
```

Consola con la ejecución:



Output - TP-5 (run)

```
run:  
Libro [titulo=Cien años de soledad, isbn=978-950-07-4200-2, Autor [nombre=Gabriel García Márquez, nacionalidad=Colombiana], Editorial [nombre=Sudamericana, direccion=Buenos Aires]]  
BUILD SUCCESSFUL (total time: 0 seconds)  
|
```

Link al repositorio:

<https://github.com/NazarenoAranda/UTN-TUPaD-P2-Nazareno-Aranda/tree/master/UTN-TUPaD-P2-Nazareno-Aranda>

Nazareno Aranda